

外部排序

数据不能全进内存时，怎样减少块 I/O 轮数

408 · 数据结构 Topic DS-12

母问题：当 I/O 远贵于比较，排序模型怎样改写？

外部排序的生成链是

Block-I/O Cost → Initial Runs → Multiway Merge → Pass Count → I/O Tradeoff.

数据先分成能装入内存的初始归并段并在内存中排序，再用 k 路归并逐轮合并。总成本主要由读写块次数和归并轮数决定，不能直接套内部排序的比较次数。

本册定位与 Owner 边界

- **Owns**：初始归并段、置换选择、多路归并、败者树、缓冲区、I/O 轮数与终止条件。
- **Uses**：Atlas Foundation 的 cost vector；存储系统的 block 概念只作为外部接口，不复制文件系统机制。
- **Stops**：内存内局部排序归 DS11；设备调度、缓存一致性和文件系统完整语义不在本册。

目录

1	初始段与多路归并	2
1.1	多路归并、缓冲区与败者树	2
1.2	归并趟数、最佳归并树与虚段	3
2	I/O 不变量与终止	3
3	范围分桶：条件满足时绕开通用归并	3
4	瓶颈由 workload 证据决定	3
5	代码与测试证据	4
6	复杂度与概念边界	6
7	做题控制协议	6

1 初始段与多路归并

设内存可容纳 M 条记录，先读入至多 M 条并排序，写成一个 **run**。归并时每个输入 **run** 保留一个当前首元素，取最小者输出并补充同一路的下一条；候选结构可用最小堆或败者树。若有 r 个初始段、每轮 k 路，轮数约为 $\lceil \log_k r \rceil$ ，每轮都要读写全部数据。普通方法每次只处理能装入内存的 M 条，所以初始段数约为 $r = \lceil N/M \rceil$ 。置换-选择排序把“选择输出”和“从输入补位”交错进行：下一条必须选择所有 **key** 不小于上一条输出值中的最小者；小于该值的记录冻结到下一 **run**。随机输入下平均 **run** 长度约为 $2M$ ，因而初始段数大致减半，但生成阶段需要一个可维护候选极值的结构，成本约为 $O(N \log M)$ 。

置换-选择实现中，内存槽位至少有“活动”和“冻结”两种状态。输出当前最小活动记录后读入新记录：若新 **key** 不小于刚输出值，继续留在本 **run**；否则标记冻结，等待 **run** 结束。活动槽位耗尽时关闭当前 **run**、把冻结集合解冻并开始下一 **run**。若没有把“冻结”与“真正输入耗尽”分开，容易提前结束或把下降 **key** 错写进当前有序段；若输入已经读完，剩余活动槽仍需全部输出。

为什么败者树值得出现

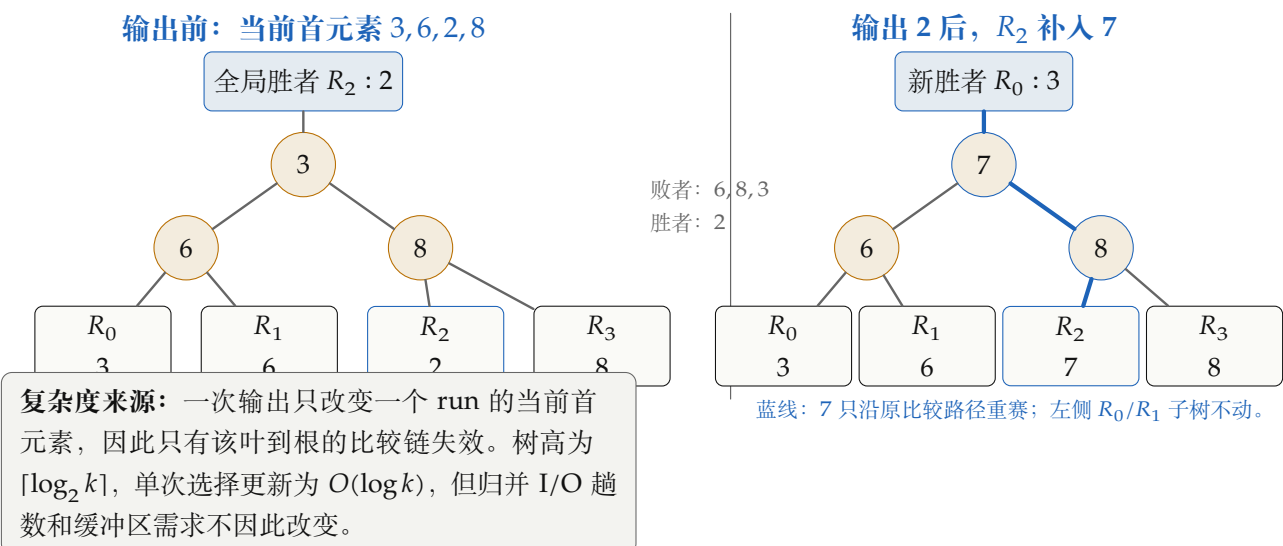
多路归并每输出一条记录都要更新“哪一路最小”。普通堆和败者树都维护候选极值；败者树显式保存比较路径，便于重复更新同一路。二者都不改变多路归并的正确性不变量。

1.1 多路归并、缓冲区与败者树

朴素地从 k 个当前首元素中找最小值，每输出一条记录要做 $k-1$ 次比较； k 增大虽然减少归并趟数，却可能让内部比较抵消 I/O 收益。败者树在内部节点保存每场比较的败者，让胜者沿路径晋级；冠军输出后只需沿该路回到根，单次调整约为 $\lceil \log_2 k \rceil$ 次比较，内部比较成本与 k 的线性增长脱钩。相比胜者树，败者树保存的正是新元素沿路径再次会遇到的对手，更新时不必重新访问兄弟节点。

4 路败者树：每次只让“刚补入的那一路”沿叶到根重新比赛

叶子只保存各 **run** 的当前首元素；内部节点保存本场败者，顶部另保留全局胜者。



图中内部节点保存当前比较的败者，顶部单独保存全局胜者；输出某一路并补入新值后，只有该叶到根的比较链失效，其他子树不需要重赛。

每一路至少需要输入缓冲区，输出还需一个输出缓冲区； k 不能脱离内存预算盲目增大。某一路耗尽时可把它的当前值替换为 $+\infty$ ，当冠军也是 $+\infty$ 才表示全部输入耗尽。初始化败者树可先让内部节点指向 $-\infty$ 虚拟叶，再逐路插入首元素，避免未初始化比较。

缓冲区设计还要区分“记录耗尽”和“缓冲块耗尽”：前者把该路从候选集合移除，后者只触发一次块读入。若采用 k 路归并而内存只有 B 个缓冲块，通常需为每路输入和输出至少保留一个块，故 k 受 B 约束；增加路数虽可降低理论趟数，却可能使每路缓冲过小、I/O 次数和管理开销上升。题目给出具体缓冲预算时，应先算可行的 k 再算 $\lceil \log_k r \rceil$ 。

1.2 归并趟数、最佳归并树与虚段

若初始段数为 r 、采用平衡 k 路归并，段数变化为 $r, \lceil r/k \rceil, \lceil r/k^2 \rceil, \dots$ ，所以趟数为 $S = \lceil \log_k r \rceil$ ；每趟读一遍写一遍，I/O 量约为 $2NS$ 。初始段长度不等时，先合并哪些段会影响它们被重复读写的次数。把段长作为叶权，归并树的总读写量等于 $2 \times WPL$ ；每次取最小的 k 个权值合并，得到 k 叉哈夫曼式最佳归并树。

严格 k 叉树要求

$$\frac{r-1}{k-1} \in \mathbb{Z}.$$

令 $u = (r-1) \bmod (k-1)$ 。若 $u \neq 0$ ，需补 $k-u-1$ 个权值为零的虚段，等价于第一次只做 $u+1$ 路归并；虚段应位于树的最底层，不能拖到最后，否则会让真实段多经历一趟 I/O。虚段只参与归并树计算，不是外存中的真实记录。

2 I/O 不变量与终止

每个 run 的当前指针只向前移动；输出序列始终是所有当前首元素中的最小值；某一路耗尽后从候选集合移除；所有输入耗尽才结束。缓冲区满/空是 I/O 事件，不是记录顺序事件，必须分开书写。块大小、内存缓冲数量和归并路数改变的是轮数与常数，不改变“每一路有序”的前提。外部排序的总时间应拆成初始段内部排序、外存读写和归并阶段内部处理三部分。在 408 的块 I/O 模型中通常假设 $t_{IO} \gg t_{cmp}$ ，所以优化第一目标是减少归并趟数和总写回量；但扩大 k 会消耗更多输入缓冲，可能降低每路缓冲效率。真实 workload 若包含 JSON 解析、复杂比较、压缩或校验，CPU 处理可能反而主导，必须测量后再定优化对象。题目给出块大小时，应按“块读/块写”计数，不把每条记录比较次数当作 I/O 次数。

3 范围分桶：条件满足时绕开通用归并

若 key 的全局范围已知，可把输入按互不重叠且有序的区间写入多个外存桶；只要每桶能在内存中排序，最后按桶区间顺序串接即可得到全局有序结果。它省去多轮通用归并，但正确性依赖三项前提：桶间范围严格有序；边界值只进入唯一桶；每桶规模不超过可处理预算。

数据倾斜会让单个热点桶溢出，此时需要二次分桶、采样后重设边界，或退回外部归并。按 Hash 值分区不能用于全局排序，因为不同 Hash 桶没有 key 顺序；它适合让相同 key 落到同一分区，从而处理去重、频次或集合交集，这些组合路线由 DS-I01 统筹。范围桶也不同于 DS11 的内存桶排序：这里的核心代价是分区写入、桶内处理与最终顺序读取的 I/O。

4 瓶颈由 workload 证据决定

统一成本模型可写为

$$T = T_{read} + T_{parse} + T_{compare} + T_{structure} + T_{write} + T_{merge}.$$

纯整数 Top-K 常常表现为读写主导；结构化记录外排可能因解析和比较让 CPU 主导。两种观察并不矛盾，它们说明“数据大”不足以决定优化方向。设计前应分别测吞吐、CPU 利用率、峰值内存、临时文件体积与各阶段耗时，再判断应增加缓冲、减少趟数、简化 key 提取，还是并行化某一阶段。

并发 worker 会同时放大输入缓冲、输出缓冲和解析对象的内存占用；串行归并与最终写回还会受 Amdahl 上限约束。线程数从 p 增到 $2p$ 不保证吞吐翻倍，尤其在设备带宽饱和、垃圾回收或共享队列竞争时。以上是工程 Extension，不改变 408 题设下按块和趟数计数的 Canonical 模型，但防止把该模型误当成所有真实系统的测量结论。

5 代码与测试证据

完整实现位于 code/ds12_external_sort.hpp，测试位于 code/ds12_external_sort_test.cpp。可执行核心用内存中的 run 模拟外部边界，输出合并结果并计算所需轮数。

```

1  #include <vector>
2
3  namespace ipara::ds12 {
4
5  inline std::vector<std::vector<int>> make_initial_runs(const std::vector<int>&
        values,
6
6                                     std::size_t
        memory_capacity) {
7  if (memory_capacity == 0) throw std::invalid_argument("memory capacity must
        be positive");
8  std::vector<std::vector<int>> runs;
9  for (std::size_t begin = 0; begin < values.size(); begin += memory_capacity)
        {
10     const std::size_t end = std::min(values.size(), begin + memory_capacity);
11     runs.emplace_back(values.begin() + static_cast<std::ptrdiff_t>(begin),
12                       values.begin() + static_cast<std::ptrdiff_t>(end));
13     std::sort(runs.back().begin(), runs.back().end());
14  }
15  return runs;
16  }
17
18  inline std::vector<int> merge_sorted_runs(const std::vector<std::vector<int>>&
        runs) {
19  struct Candidate { int value; std::size_t run; std::size_t index; };
20  auto greater = [](const Candidate& left, const Candidate& right) {
21      return left.value > right.value;
22  };
23  std::priority_queue<Candidate, std::vector<Candidate>, decltype(greater)>
        pending(greater);
24  std::size_t total = 0;
25  for (std::size_t run = 0; run < runs.size(); ++run) {
26      total += runs[run].size();
27      if (!runs[run].empty()) pending.push(Candidate{runs[run].front(), run, 0});
28  }
29  std::vector<int> output;
30  output.reserve(total);

```

```

31 while (!pending.empty()) {
32     const Candidate current = pending.top();
33     pending.pop();
34     output.push_back(current.value);
35     const std::size_t next = current.index + 1;
36     if (next < runs[current.run].size()) {
37         pending.push(Candidate{runs[current.run][next], current.run, next});
38     }
39 }
40 return output;
41 }
42
43 inline std::size_t merge_passes(std::size_t run_count, std::size_t fan_in) {
44     if (fan_in < 2) throw std::invalid_argument("fan-in must be at least two");
45     std::size_t passes = 0;
46     while (run_count > 1) {
47         run_count = (run_count + fan_in - 1) / fan_in;
48         ++passes;
49     }
50     return passes;
51 }
52
53 } // namespace ipara::ds12
54
55 #endif

```

代码清单 1: 有序 runs 的多路归并与轮数估算

```

1 using ipara::ds12::merge_passes;
2 using ipara::ds12::merge_sorted_runs;
3
4 int main() {
5     const std::vector<int> input{9, 1, 7, 3, 5, 2, 8, 4, 6};
6     const auto runs = make_initial_runs(input, 3);
7     assert((runs == std::vector<std::vector<int>>{{1, 7, 9}, {2, 3, 5}, {4, 6,
8         8}}));
9     assert((merge_sorted_runs(runs) == std::vector<int>{1, 2, 3, 4, 5, 6, 7, 8,
10         9}));
11     assert((merge_sorted_runs({}, {1, 4}, {}, {2, 3}) == std::vector<int>{1, 2,
12         3, 4}));
13     assert(merge_passes(1, 2) == 0);
14     assert(merge_passes(10, 3) == 3);
15     bool capacity_error = false;
16     try { (void)make_initial_runs(input, 0); } catch (const std::invalid_argument
17         &) { capacity_error = true; }
18     assert(capacity_error);
19     bool fan_in_error = false;
20     try { (void)merge_passes(2, 1); } catch (const std::invalid_argument&) {
21         fan_in_error = true; }
22     assert(fan_in_error);
23 }

```

代码清单 2: 空 run、单 run 与多路耗尽测试

```
1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/  
   ds12_external_sort_test.cpp -o /tmp/ds12_test  
2 /tmp/ds12_test
```

6 复杂度与概念边界

阶段	比较/更新	I/O 视角	不变量	边界
初始段	每段内存排序	读写 N	每段有序	$N \leq M$
k 路归并	每条更新候选	每轮读写 N	各路首元素候选	空 run
总过程	依赖实现	$O(N \log_k r)$ 轮次量级	指针单调前进	输入耗尽

7 做题控制协议

先标 N, M, k, r ，再算初始段数量和归并轮数；每一轮分别写“读、比较、输出、补位、写”。看到全部数据能进内存，停止套外部 I/O 公式并转 DS11；看到块大小或缓冲区变化，重新计算轮数而非只比较关键字次数。

压缩复原

五问：内存一次容纳多少？初始 run 怎样形成？当前候选来自几路？哪一路耗尽如何退出？总成本按比较还是块 I/O 计？